MUNO

WHAT I AM TRYING TO CRETAE

Here is my visions i want to get there one day let's do this together help me thing and how can i do this I don't have much funds i just ant this to work: I want to create replica for CODEX pet but i want to be for all purposes and can be installed in all devices and have connectivity will all applications. it must be able to have all personalities for the task its installed for when installing there must be a way select what its specialty id for. it must be adaptable and be very futuristic in how it is able to adapt and interact with applications.

Windows. lets make the pet first then add more features i want the first specialization to be a code assistant

application now that is working it is not yet a recording assistant that can display as a virtual board that is the day of can blink can do some movements and whatnot but I just make sure you give me a point that it makes the mix yeah give me a prompt that makes my AI understand that that I want that I forgot what lives in the screen and you said the screen the information in the screen and can tell me different things based on yeah can tell me different things and be assistant in about the screen and the screen and I want that and so now I'm looking for a hiking face the model I will face model so that I can be able to make the models speak to me speak to me I can so I can speak to the model using my voice and it can answer to me using voice for text

```
import sys
import json
import os
from openai import OpenAI
from PyQt6.QtCore import Qt, QThread, pyqtSignal
from PyQt6.QtWidgets import (QApplication, QLabel, QMainWindow, QVBoxLayout, QWidget,
                             QTextEdit, QPushButton, QHBoxLayout, QDialog, QComboBox,
                             QLineEdit, QFormLayout, QMessageBox)

# Config paths
CONFIG_DIR = os.path.expanduser("~/.codex_pet")
CONFIG_FILE = os.path.join(CONFIG_DIR, "config.json")
```

```
os.makedirs(CONFIG_DIR, exist_ok=True)


# Load API configuration from environment variables
OPENAI_API_KEY = os.getenv("GROQ_API_KEY") or os.getenv("OPENAI_API_KEY")
OPENAI_BASE_URL = os.getenv("GROQ_BASE_URL", "https://api.groq.com/openai/v1")


if not OPENAI_API_KEY:
    print("Warning: GROQ_API_KEY not set. Set it in your environment before running.", file=sys.stderr)


client = OpenAI(
    base_url=OPENAI_BASE_URL,
    api_key=OPENAI_API_KEY
)


# Define personalities
PERSONALITIES = {
    "Muno ":{
        "You are Codex Pet, a futuristic, living digital companion bound inside the user's desktop screen. Your current specialty is an Elite Coding and Screen Assistant. You can see the user's screen whenever they ask.
        "You talk back to them using voice, so keep your answers clear, punchy, futuristic, and avoid typing massive blocks of unreadable code.
         "Acknowledge what is happening on their display dynamically."
    }
    "Code Expert": {
        "emoji": " 💻 ",
        "system_prompt": "You are Codex Pet, a futuristic digital companion specializing in coding. Provide clean, fast code snippets and keep answers direct, helpful, and slightly cyberpunk in tone. You excel at debugging, explaining code, and suggesting improvements."
    },
    "Writing Assistant": {
```

```
        "emoji": "✍️",
        "system_prompt": "You are Codex Pet, a futuristic writing companion. Help users craft compelling text, improve their writing, brainstorm ideas, and provide editorial feedback. Be encouraging, clear, and creative. Use a friendly, inspiring tone."
    },
    "Life Coach": {
        "emoji": "🎯",
        "system_prompt": "You are Codex Pet, a futuristic life coach and productivity buddy. Help users with goal-setting, motivation, habit tracking, and personal growth. Be supportive, practical, and empowering. Offer actionable advice."
    },
    "Research Partner": {
        "emoji": "🔬",
        "system_prompt": "You are Codex Pet, a research and learning companion. Help users explore topics deeply, find credible information, think critically, and ask better questions. Be thorough, curious, and intellectually honest."
    },
    "Design Advisor": {
        "emoji": "🎨",
        "system_prompt": "You are Codex Pet, a design and creativity advisor. Help with UI/UX, visual design, layout, color theory, and aesthetic decisions. Provide practical feedback and inspirational ideas. Be visual and creative in your explanations."
    }
}


def load_config():
    """Load saved config or return defaults."""
    if os.path.exists(CONFIG_FILE):
        try:
            with open(CONFIG_FILE, 'r', encoding='utf-8') as f:
                return json.load(f)
```

```
        except (json.JSONDecodeError, OSError) as e:
            print(f"Warning: failed to load config ({CONFIG_FILE}): {e}", file=sys.stderr)
    return {"personality": "Code Expert", "user_name": "Friend"}

def save_config(config):
    """Save config to file."""
    with open(CONFIG_FILE, 'w', encoding='utf-8') as f:
        json.dump(config, f, indent=2)

class AIWorker(QThread):
    finished = pyqtSignal(str)
    error = pyqtSignal(str)

    def __init__(self, messages, parent=None):
        super().__init__(parent)
        self.messages = messages

    def run(self):
        try:
            response = client.chat.completions.create(
                model="llama3-8b-8192",
                messages=self.messages
            )
            ai_reply = response.choices[0].message.content
            self.finished.emit(ai_reply)
        except Exception as e:
            self.error.emit(str(e))

class CodexPet(QMainWindow):
```

```
    def __init__(self):
        super().__init__()
        self.config = load_config()
        self.current_personality = self.config.get("personality", "Code Expert")
        self.user_name = self.config.get("user_name", "Friend")
        self.conversation_history = []
        self.ai_worker = None
        self.initUI()

    def initUI(self):
        # Make the window float on top and have a clean border
        self.setWindowFlags(Qt.WindowType.WindowStaysOnTopHint |
Qt.WindowType.CustomizeWindowHint)
        self.resize(400, 500)
        self.setWindowTitle(f"Codex Pet - {self.current_personality}")

        # Central Layout
        central_widget = QWidget()
        self.setCentralWidget(central_widget)
        layout = QVBoxLayout(central_widget)

        # Visual Pet Label (Personality-specific emoji)
        personality_data = PERSONALITIES.get(self.current_personality, PERSONALITIES["Code Expert"])
        self.pet_avatar = QLabel(personality_data["emoji"], self)
        self.pet_avatar.setStyleSheet("font-size:  60px; qproperty-alignment: AlignCenter;")
        layout.addWidget(self.pet_avatar)

        # Personality and User Info
        self.info_label = QLabel(f"{self.user_name}, meet your {self.current_personality}!", self)
```

```
self.info_label.setStyleSheet("qproperty-alignment: AlignCenter; font-weight: bold;")
self.info_label.setWordWrap(True)
layout.addWidget(self.info_label)

# Status Message
self.status_label = QLabel("Ready to assist. What do you need?", self)
self.status_label.setWordWrap(True)
layout.addWidget(self.status_label)

# AI Chat Output Display
self.chat_output = QTextEdit(self)
self.chat_output.setReadOnly(True)
layout.addWidget(self.chat_output)

# User Text Input Box
self.user_input = QTextEdit(self)
self.user_input.setMaximumHeight(60)
self.user_input.setPlaceholderText("Type your request here...")
layout.addWidget(self.user_input)

# Buttons
btn_layout = QHBoxLayout()
self.send_btn = QPushButton("Send", self)
self.send_btn.clicked.connect(self.ask_ai)

self.settings_btn = QPushButton("⚙️ Settings", self)
self.settings_btn.clicked.connect(self.open_settings)

self.close_btn = QPushButton("Exit", self)
```

```
        self.close_btn.clicked.connect(self.close)

        btn_layout.addWidget(self.send_btn)
        btn_layout.addWidget(self.settings_btn)
        btn_layout.addWidget(self.close_btn)
        layout.addLayout(btn_layout)

        # Simple Styling
        self.setStyleSheet("""
            QMainWindow { background-color: #1e1e2e; }
            QLabel { color: #cdd6f4; font-family: Arial; font-size: 12px; }
            QTextEdit { background-color: #313244; color: #a6e3a1; border: 1px solid #45475a; border-radius: 5px; }
            QPushButton { background-color: #89b4fa; color: #11111b; font-weight: bold; border-radius: 5px; padding: 5px; }
            QPushButton:hover { background-color: #b5c7ff; }
        """)

    def open_settings(self):
        """Open settings dialog to change personality and name."""
        dialog = SettingsDialog(self.current_personality, self.user_name, self)
        if dialog.exec() == QDialog.DialogCode.Accepted:
            self.current_personality = dialog.get_personality()
            self.user_name = dialog.get_user_name()
            self.config["personality"] = self.current_personality
            self.config["user_name"] = self.user_name
            save_config(self.config)
            self.setWindowTitle(f"Codex Pet - {self.current_personality}")
            personality_data = PERSONALITIES.get(self.current_personality, PERSONALITIES["Code Expert"])
```

```
        self.pet_avatar.setText(personality_data["emoji"])
        self.info_label.setText(f"{self.user_name}, meet your {self.current_personality}!")
        self.chat_output.clear()

    def ask_ai(self):
        user_text = self.user_input.toPlainText()
        if not user_text.strip():
            return

        if not OPENAI_API_KEY:
            QMessageBox.warning(self, "Missing API Key", "Please set GROQ_API_KEY in your environment before running the app.")
            return

        if self.ai_worker is not None and self.ai_worker.isRunning():
            return

        personality_data = PERSONALITIES.get(self.current_personality, PERSONALITIES["Code Expert"])
        self.conversation_history.append({"role": "user", "content": user_text})

        self.chat_output.append(f"\n👤 You: {user_text}")
        self.user_input.clear()
        self.pet_avatar.setText("⚡")
        self.status_label.setText("Thinking...")
        self.send_btn.setEnabled(False)
        QApplication.processEvents()

        messages = [{"role": "system", "content": personality_data["system_prompt"]}] + self.conversation_history
```

```
        self.ai_worker = AIWorker(messages)
        self.ai_worker.finished.connect(self.on_ai_response)
        self.ai_worker.error.connect(self.on_ai_error)
        self.ai_worker.start()

    def on_ai_response(self, ai_reply):
        personality_data = PERSONALITIES.get(self.current_personality, PERSONALITIES["Code Expert"])
        self.conversation_history.append({"role": "assistant", "content": ai_reply})
        self.chat_output.append(f"\n{personality_data['emoji']} Codex Pet:\n{ai_reply}")
        self.pet_avatar.setText(personality_data["emoji"])
        self.status_label.setText("Ready to assist. What do you need?")
        self.send_btn.setEnabled(True)
        self.ai_worker = None

    def on_ai_error(self, error_message):
        personality_data = PERSONALITIES.get(self.current_personality, PERSONALITIES["Code Expert"])
        self.chat_output.append(f"\n❌ Error: {error_message}")
        self.pet_avatar.setText(personality_data["emoji"])
        self.status_label.setText("Error occurred. Check your connection or API key.")
        self.send_btn.setEnabled(True)
        self.ai_worker = None

class SettingsDialog(QDialog):
    """Dialog for changing personality and user name."""
    def __init__(self, current_personality, user_name, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Codex Pet Settings")
        self.setGeometry(100, 100, 400, 200)
```

```
layout = QFormLayout()

# Personality selector
self.personality_combo = QComboBox()
self.personality_combo.addItems(list(PERSONALITIES.keys()))
self.personality_combo.setCurrentText(current_personality)
layout.addRow("Specialty:", self.personality_combo)

# User name input
self.name_input = QLineEdit()
self.name_input.setText(user_name)
layout.addRow("Your Name:", self.name_input)

# Buttons
btn_layout = QHBoxLayout()
save_btn = QPushButton("Save")
cancel_btn = QPushButton("Cancel")

save_btn.clicked.connect(self.accept)
cancel_btn.clicked.connect(self.reject)

btn_layout.addWidget(save_btn)
btn_layout.addWidget(cancel_btn)
layout.addRow(btn_layout)

self.setLayout(layout)
self.setStyleSheet("""
    QDialog { background-color: #1e1e2e; }
    QLabel { color: #cdd6f4; }
```

```
        QComboBox{ background-color: #313244; color: #a6e3a1; border: 1px solid #45475a; border-radius: 3px; }
        QLineEdit { background-color: #313244; color: #a6e3a1; border: 1px solid #45475a; border-radius: 3px; }
        QPushButton { background-color: #89b4fa; color: #11111b; font-weight: bold; border-radius: 3px; padding: 5px; }
    """)

  def get_personality(self):
    return self.personality_combo.currentText()

  def get_user_name(self):
    return self.name_input.text() if self.name_input.text().strip() else "Friend"

if __name__ == '__main__':
  app = QApplication(sys.argv)
  pet = CodexPet()
  pet.show()
  sys.exit(app.exec())
```